

FIFA 14 Local FUT: Custom Card Designs

What was changed to add new card types, how it works, and how to maintain it yourself

Project C:\Games\FIFA-14-Local-FUT-main

Game C:\Program Files\EA Games\FIFA 14\Game

State as of 25 Sep 2026

15 new card types

rareFlag 14 to 28 each get their own background in every view: bio (large), reserves/subs (collapsed), squad pitch (small) and preview ("fake") cards. Up to 16 fit.

Legend slot unlocked

rareFlag 12 draws `cards_bg_70`, a slot EA's code already asks for but never shipped art for on PC.

Own text colors

Rating, name and stats colors per type, for up to 4 new types (now 14, 15, 16, 18).

No rare glare

New types never get the animated yellow rare shine, odd rareFlag or not.

The short version

You edit one file, `artifacts\fut-card-slots.json`, and add texture files. When the game starts, the normal launcher turns that file into bytecode edits of the card UI scripts, checks them, and installs them. Original game data is never overwritten, so every change can be undone.

Contents

1. Everyday use

- 1.1 The file you edit
- 1.2 Add a design for a new type
- 1.3 Give a player a card type
- 1.4 Change text colors
- 1.5 Add or remove types
- 1.6 Undo and switch off
- 1.7 What happens at launch

2. Files and tools reference

3. How it works

- 3.1 From a number to a picture
- 3.2 Why the UI scripts, not the .exe

3.3 How edits get into the archives

3.4 Apt bytecode in two minutes

3.5 Rewriting code without moving it

3.6 New designs: the four switches

3.7 Legend slot, texture pre-cache

3.8 Text colors

3.9 Glare removal

4. Limits, gotchas and lessons

Appendix A: rareFlag to texture ids

Appendix B: all rewritten regions

Appendix C: opcode cheat sheet

Appendix D: commands and glossary

1. Everyday use

1.1 The one file you edit

C:\Games\FIFA-14-Local-FUT-main\artifacts\fut-card-slots.json describes every custom type. Its current content (the `_help` text is left out here):

```
{
  "new_types": [14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28],
  "precache": [14, 15, 16],
  "text_colors": {
    "14": {"header": "#0B2A6F", "text": "#0B2A6F"},
    "15": {"header": "#E6E6E6", "text": "#E6E6E6"},
    "16": {"header": "#FF10F0", "text": "#E6E6E6"},
    "18": {"header": "#E6E6E6", "text": "#E6E6E6"}
  }
}
```

Key	Meaning	Limits
<code>new_types</code>	rareFlag values that get their own design. rareFlag N uses texture ids N , $N+1000$, $N+100$ and $N+300$ (see 1.2).	Values 14 to 49; at most 16 types.
<code>precache</code>	Types whose textures are loaded together with EA's own backgrounds when the hub or squad opens. The others load the first time a card needs them, which also works.	At most 4; each must also be in <code>new_types</code> .
<code>text_colors</code>	Optional per type. header = rating (and position on big cards); text = name and other texts; footer = stats row on big cards (defaults to text). Colors are <code>#RRGGBB</code> .	At most 4 types. Types without an entry keep the standard common (even) / rare (odd) colors.

If you break a limit or make a typo, the launcher prints *"fut-card-slots.json could not be turned into edits"* and keeps the previous working edits. Nothing gets half-applied. To see the exact reason, run the generator by hand (Appendix D).

1.2 Recipe: add a design for a new type

- 1 **Make four textures** for rareFlag N . All are DDS with **DXT5 (BC3)** compression and **no mipmaps**. The file sizes are an easy check that the export settings were right.

Where it shows	File (inside <code>itemBGs\</code>)	Pixels	File size
Bio / big card	<code>large\cards_bg_<N>.dds</code>	256 x 256	65,664 bytes
Reserves and subs (collapsed)	<code>collapsed\cards_bg_<N+1000>.dds</code>	128 x 256	32,896 bytes
Squad pitch (base layer)	<code>small\cards_bg_<N+100>.dds</code>	128 x 128	16,512 bytes
Squad pitch (second layer)	<code>small\cards_bg_<N+300>.dds</code>	128 x 128	16,512 bytes

Folder: C:\Program Files\EA Games\FIFA 14\Game\data\ui\external\ion_fut\imgAssets\itemBGs\ . Copying into Program Files needs admin rights (accept the UAC prompt in Explorer). Example for rareFlag 20: `cards_bg_20`, `cards_bg_1020`, `cards_bg_120`, `cards_bg_320`.

- 2 **Check that N is in `new_types`**. 14 to 28 already are, so for those you only add art. Optionally add a `text_colors` entry.
- 3 **Give a player that rareFlag** (section 1.3).
- 4 **Start the game the normal way**. Restart it if it was running. That's the reliable way for new files to be picked up: during testing, a texture that was missing wasn't looked for again in the same session.

Art status today: 14, 15, 16, 17, 18 and 19 have full sets. 70 (the Legend slot) holds World Cup stand-in copies. 20 to 28 are wired but have no art yet. A type with no art shows a plain white "empty slot" rectangle. If you see that, check the file names.

1.3 Recipe: give a player a card type

The local server keeps every item in a SQLite database and sends each item to the game as JSON. The card type is that JSON's rareFlag.

- Database: `C:\Users\nouve\AppData\Local\FIFA14LocalFUTBeta\local-fut-beta-v2410.sqlite3`
- Table `items`, column `payload` (JSON). Set **both** keys, `"rareflag"` and `"rareFlag"`, to the same number.
- Find the row by `item_id` or `asset_id`. To go from squad slot to item, table `squad_players` has `slot_index` and `item_id`. The `name` inside the payload can be stale (one player showed as a different player's name), so trust ids over names.
- The `squad_players.rare_flag` column is **not** what the game uses. Editing it does nothing.

Any SQLite editor works, for example DB Browser for SQLite: edit the payload cell, then choose *Write Changes*. The game picks it up the next time it fetches your club (leave the squad and open it again; restart if it doesn't change). A backup of the original squad rareFlags is in `artifacts\rareflag_backup_2026-09-23.json`. Abate is on 9 (FUTTIES) on purpose, not from a test.

1.4 Recipe: change text colors

Edit or add a `text_colors` entry and start the game. For example, `"18": {"header": "#E6E6E6", "text": "#E6E6E6"}` gives 18 the same white EA uses on rare and MOTM cards. The four colored types are all in use now (14, 15, 16, 18). To color another type, remove one of them first.

Your types	Header (rating, position)	Text (name, other texts)
14	 #0B2A6F dark blue	 #0B2A6F dark blue
15	 #E6E6E6 white	 #E6E6E6 white
16	 #FF10F0 neon pink	 #E6E6E6 white
18	 #E6E6E6 white	 #E6E6E6 white

EA's own palette, for picking similar colors: rare (all tiers)  #E6E6E6 · common gold  #38311A, silver  #242424, bronze  #26160F · TOTW gold  #FFEE31, silver  #DADADA, bronze  #D1B0A0 · Legend  #9E7F00.

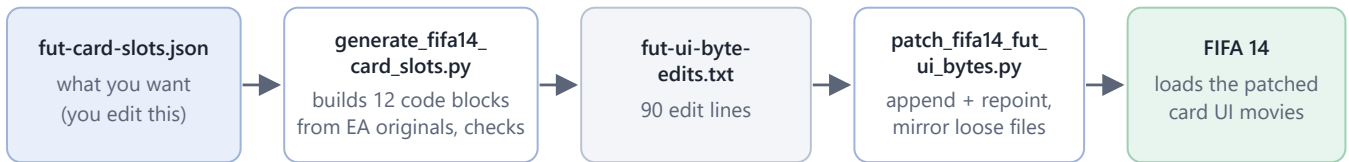
1.5 Recipe: add or remove types

- **Add:** put the number in `new_types` (14 to 49, at most 16 types in total) and add its art. The list doesn't need to be continuous.
- **Remove:** take it out of `new_types` (and out of `precache` / `text_colors`). Players with that rareFlag go back to EA's fallback look: odd means rare (with the shine), even means common.

1.6 Undo and switch off

- **Everything off, back to the original EA card code:** rename `fut-card-slots.json` (for example to `fut-card-slots.off.json`). Then replace the contents of `artifacts\fut-ui-byte-edits.txt` with a single comment line such as `# off`, and start the game once. The patch tool points all six archive entries back at the untouched original records and restores the three loose UI files from their backups. (Tested as a dry run: 6 entries repointed, 3 files restored.)
- **Back on:** rename the JSON back and start the game. The edits file is regenerated automatically.
- **Manual last resort:** full copies of the original archives and indexes sit next to them as `patch.big.uibytes.bak`, `patch.bh.uibytes.bak`, `cards0.big.uibytes.bak`, `cards0.bh.uibytes.bak`. The loose `card.big`, `cardsmall.big` and `futmain.big` have `.uibytes.bak` copies too.

1.7 What happens at launch



All of this runs inside `tools\run_fifa14_local_beta.ps1`, after the project's spinner patcher, before the game starts.

Figure 1. Your config becomes installed edits on every launch. If generation fails, the previous edits file is applied unchanged.

The order matters. An older project patch, the FUT spinner fix (`patch_fifa14_fut_spinner.py`), rebuilds `patch.bh` from a snapshot on every launch, which erases other changes to that index. So the card edits must be installed *after* it, on every launch. That's why this is a launcher step and not a one-time install. The step is non-fatal: if it fails, the game still starts, just with original cards.

2. Files and tools reference

File	What it is / how to use it
<code>artifacts\fut-card-slots.json</code>	Your config. The only file you normally touch (section 1).
<code>tools\generate_fifa14_card_slots.py</code>	The generator. Reads the config and the original EA UI movies, then assembles 12 replacement code blocks (Appendix B). It checks every block and writes the edits file. It refuses to run if the original files aren't byte-for-byte what it expects (SHA-256 check), so a game update can't be silently mis-patched.
<code>artifacts\fut-ui-byte-edits.txt</code>	Generated edit list, one line per edit: <code>inner file : offset : old bytes : new bytes</code> . Marked GENERATED; it's rebuilt from the config on every launch, so don't edit it by hand except to switch everything off (1.6).
<code>tools\patch_fifa14_fut_ui_bytes.py</code>	The installer. Applies the edits inside <code>patch.big</code> and <code>cards0.big</code> and mirrors them onto the loose UI copies. Dry run by default; <code>--apply</code> writes (game must be closed, admin rights needed). Other flags: <code>--no-loose</code> , <code>--archives</code> , <code>--edit</code> (single edit), <code>--state-file</code> .
<code>artifacts\fut-ui-bytes-state.json</code>	The installer's memory: where each original record is and where the patched copies were appended. This lets it re-apply or revert by changing only an index entry, without growing the archive.
<code>tools\run_fifa14_local_beta.ps1</code>	The launcher script. Runs generator, then installer, right after the vp6 intro check.
<code>*.uibytes.bak</code> (next to the archives and the loose UI files)	Untouched originals, created before the first write.
<code>artifacts\rareflag_backup_2026-09-23.json</code>	The squad's original rareFlags from before testing.
<code>...\imgAssets\itemBGs\{large,collapsed,small}\</code>	The card background textures, EA's and yours.
<code>...\FIFA14LocalFUTBeta\local-fut-beta-v2410.sqlite3</code>	The local server's database (items and their rareFlags).

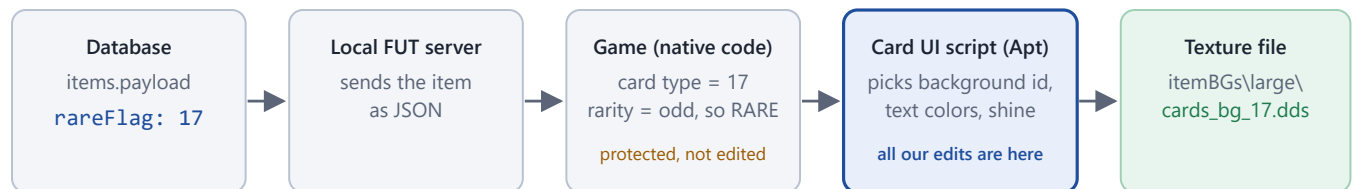
Game files involved

UI movie	What it draws	Where the game reads it
<code>card.big</code>	The big card (bio) and the collapsed card (reserves/subs), plus the "fake" preview variant (<code>CardFake</code>)	Packed in <code>patch.big</code> (wins) and <code>cards0.big</code> ; loose copy in <code>data\ui\external\ion_fut\components\</code>
<code>cardsmall.big</code>	The small card on the squad pitch, plus its fake variant	Same three places
<code>futmain.big</code>	Shared FUT helpers: the text-color palette and the texture pre-cache	Same, in <code>...\ion_fut\main\</code>

All three are patched in every place they exist, so it never matters which copy a screen reads.

3. How it works

3.1 From a number in the database to a picture on the card



The key finding: the native code passes any rareFlag of 14 or more through unchanged, so every decision we needed lives in the UI script.

Figure 2. The path from database value to texture.

Every item has a **rareFlag**. The game's native code stores it as the card's *type* (EA calls this field `CARD_TOTW`) and derives a *rarity* from it. For anything that isn't one of EA's special types, odd means rare and even means common. The card itself is drawn by a UI **movie** in EA's Flash-like format, **Apt**. A movie holds graphics plus a small program (**bytecode**), and that program decides which background number to load, which colors the texts get, and whether the rare shine plays.

The original program only knew the types 3 to 13 (TOTW, the colored promos, TOTS, Legend, World Cup). Any other value fell through to the plain common or rare look by parity. That's why rareFlag 14 used to look common and 15 rare. A test confirmed the native code doesn't filter values of 14 and above. So teaching the *UI program* about new numbers was enough, and no native code had to change.

3.2 Why the UI scripts, not the game executable

`fifa14.exe` and `CardsDLLzf.dll` are protected by anti-tamper code. Changing or hooking their code made the game die within minutes, and the bytes on disk don't even match what runs in memory. Read-only techniques (CPU hardware breakpoints, tracing which files the game opens) were used only to investigate, never left active. The Apt movies, by contrast, are plain data files the game loads and interprets. Changing them is like changing a level file, not the engine, and the protection doesn't check them.

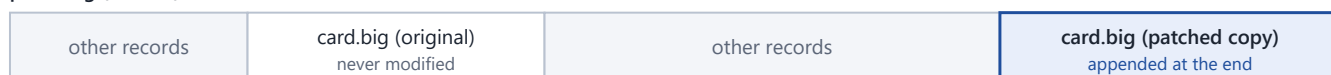
3.3 How edits get into the game's archives

The movies are packed inside the archives `patch.big` and `cards0.big`. Each archive has an **index** (`.bh`) listing, for every inner file, where its **record** starts and how long it is. `patch.big` overrides `cards0.big`. Records are compressed in independent chunks (EA's "chunkzip"). Two facts shaped the design:

- A recompressed record is always a bit bigger than EA's original (their compressor is better than zlib's best setting), so it can't be written back in place.

- The spinner patcher restores `patch.bh` from a snapshot on every launch. If a record had been overwritten in place with a different size, that restore would point at broken data.

patch.big (archive)



patch.bh (index)



patched: point at the copy (8 bytes change)
revert / spinner restore: point at the original

Figure 3. Append and repoint. The original data is untouched, so reverting or the spinner's restore always lands on valid data.

So the installer **appends** the patched record at the end of the archive, recompressing only the chunks that changed, and **repoints** just that one index entry. It remembers each appended copy in its state file. Later launches with the same edits only rewrite the 8-byte index entry, so the archive doesn't keep growing. Removing an edit simply points the entry back at the original. The installer also mirrors the same edits onto the loose copies in `data\ui\` (keeping `.uibytes.bak` backups), so every copy of a movie the game could read is identical.

3.4 Apt bytecode in two minutes

Apt's program format is a close cousin of Flash's ActionScript bytecode (AVM1), with EA-specific shortcuts. It's a **stack machine**. Instructions push values onto a stack; operations take their inputs off the stack and push the result. Names like `mTOTW` are stored once in a **constant table** and referenced by index. Each function also has numbered **registers** (`r0`, `r1` ...), fast local variables. Everything below had to be worked out from the files, since EA never published the format.

```
; "is this card purple (type 4)?" in two ways
; EA's original style: look up the enum constant by name every time
b9 00      push_reg 0          ; r0 holds mTOTW
b9 01      push_reg 1          ; r1: where the Card class lives
af f4      member 'Card'
a3 55 01   push 'eCardTOTW'
4e         GetMember
af 7c      member 'PURPLE_CARD' ; the enum's value (4)
66         StrictEquals
9d .. ..   IfTrue -> target    ; padding + 4-byte offset

; compact style used by the generator: compare with a literal
b9 00      push_reg 0
b5 04      push_byte 4
66         StrictEquals
9d .. ..   IfTrue -> target
```

Two details made the format hard to crack. **Branch and register instructions** (`0x9D` `IfTrue`, `0x99` `Jump`, `0x87` `StoreRegister`) put their 4-byte operand on the next 4-byte boundary of the file offset, with 0 to 3 zero padding bytes first. The branch target is *operand address* + 4 + *offset*. And **1-byte name operands** index a small per-code-block table, not the global one. In `card.big` the two happen to coincide; in `cardsmall.big` and `futmain.big` they don't. Appendix C lists the opcodes used.

3.5 Rewriting code without moving it

Inserting bytes into a function would shift everything after it and invalidate size fields and branch offsets throughout the movie. So **every change is a same-size rewrite of one region**: a stretch of an existing function is replaced by new code that does the original job plus the new behavior, and whatever space is left is filled with never-executed `Pop` bytes. Nothing outside the region moves.

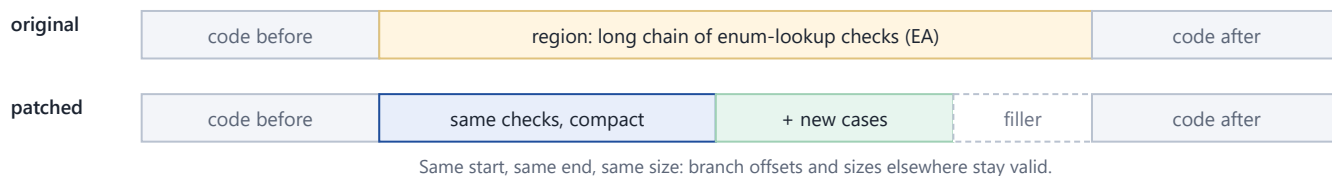


Figure 4. A same-size region rewrite.

The room comes from EA's verbose style: an enum-lookup check costs about 20 bytes, a literal compare about 11 (see 3.4). The pre-cache function also got room by dropping four debug `trace()` calls, which do nothing in the retail game. The generator is a small **assembler** (labels, branches, alignment padding) with built-in safety checks, and it refuses to write anything that fails one:

- the original movie matches a known SHA-256;
- every piece of EA code it copies is compared against the exact expected bytes;
- the whole patched function decodes cleanly from start to end;
- every branch lands exactly on the start of an instruction;
- no code outside a region jumps into the middle of it.

3.6 New designs: the four background switches

Four functions choose a card's background number, one per view. All four had to learn the new types:

Function (movie)	Draws	Texture ids for type <i>N</i>
<code>Card::setBackgroundImage</code> (card.big)	Bio (large) and reserves/subs (collapsed)	<i>N</i> and <i>N</i> +1000
<code>CardFake::_SetFakeBackground</code> (card.big)	Preview ("fake") big cards, built straight from item data	<i>N</i>
<code>CardSmall::setBackgroundImage</code> (cardsmall.big)	Squad pitch	<i>N</i> +100 and a second layer <i>N</i> +300
<code>CardSmall::_SetFakeBackground</code> (cardsmall.big)	Preview small cards	<i>N</i> +100 (one layer)

Each one computes a default id from tier and rarity (for the big card: common 1 to 3, rare 11 to 13, TOTW 51 to 53) and then runs a *switch* on the card type to override it for special cards. The rewrite keeps every EA case and adds the new ones:

```
// Card::setBackgroundImage switch after the rewrite (id ends up in register r2)
r0 = mTOTW                                // the card type (= rareFlag)
if r0 == 11      r2 = mLevel + 66          // TOTS: 67 / 68 / 69 by tier
if r0 == 12      r2 = 70                   // Legend
if r0 == 4 ... 10 r2 = r0 + 56              // purple 60 ... teal 66 (one compare each)
if r0 == 14 ... 28 r2 = r0                // NEW: your types (one compare each)
otherwise        keep the default          // common / rare / TOTW
```

The +1000 / +100 / +300 steps aren't our invention. They're the offsets EA's code already adds for the collapsed size and the small card's two layers (`COLLAPSED_BG_OFFSET`, EA's spelling, `SMALL_BG_ID_OFFSET` and `FOUR_ROWS_BG_ID_OFFSET`). Picking ids = *N* lets new types reuse that arithmetic. **Why 14 to 49:** the small ids *N*+100 and *N*+300 must stay below TOTW's small ids 151 and 351, and every number in that range is unused by EA. **Why 16 types at most:** the fake big-card switch region is the smallest, and it's full at 16 compares.

3.7 The Legend slot and the texture pre-cache

Legend (rareFlag 12): EA's code already maps Legend cards to id 70, but the PC game never shipped `cards_bg_70`. Tracing file opens showed exactly four failed requests every time: `cards_bg_70`, `1070`, `170` and `370`. Putting art there makes Legend cards use it, with no code change. The current files are World Cup stand-ins.

Pre-cache: when the hub or the squad opens, `FutHelpers::_CacheItemBGImages` (futmain.big) loads about 95 known background textures in one go, from three hard-coded lists. The generator rebuilds that function with up to 4 of your types added. Types not in the list load on demand the first time a card needs them, which was checked for the bio and the pitch. Pre-caching just avoids a short pop-in.

3.8 Text colors

Three functions choose text colors: `Card::SetMainFontColors` (rating and position, name and bottom line of big and collapsed cards), `Card::setFooterFontColor` (the stats row on big cards) and the small card's color function (rating, everything else). They pick from EA's palette object `gFutHelpers.FUT_TEXT_COLOR` in a fixed order. The rewrite adds your types into that order, *after* the World Cup check so World Cup cards keep their white:

```
if type === 3          TOTW palette (by tier)
if type === 12         Legend gold
if World Cup item     white
if type === 14         header #0B2A6F, text #0B2A6F    // from your config
... (one entry per colored type)
otherwise             canShine ? RARE palette : COMMON palette    (by tier)
```

Your colors are stored as raw numbers in the code (opcode `0xB7` pushes a 32-bit value, `0x00RRGGBB`). **Why 4 at most:** the footer function's region is the smallest of the three. To make room there, the rewrite keeps the tier and the palette object in spare registers instead of fetching them again and again.

3.9 Glare removal

The rare "glare" is a separate layer on the card, `mcRareShine`, animated by a timer (`startShineInterval`, then the animation stages 1 to 3). The timer only plays the animation while that layer is **visible**. The layer is made visible in exactly four places: the start of `setBackgroundImage` in `Card` and `CardSmall`, and the end of both fake variants. Each does "special card or rare card, so show the shine" (the fake variants: any `rareFlag` above 0). Because odd `rareFlags` count as rare, `rareFlag` 17 got the glare and 16 didn't.

```
// start of Card::setBackgroundImage after the rewrite
r0 = mTOTW
if 13 < r0 and r0 < 29      goto OFF      // NEW: your types never shine
if 2 < r0 and r0 < 13      r3 = TOTW offset, goto ON    // TOTW ... Legend
if rarity is not RARE      goto OFF
r3 = RARE offset
ON: reset animation; mcRareShine._visible = true; start timer
    if not a World Cup item, done
OFF: mcRareShine._visible = false; stop timer
```

Only the *shine switch* changed. The separate "can this card shine?" test (`canCardShine` / `canShine`) was left alone on purpose, because it also chooses the rare text colors and the selection glow. That's why `rareFlag` 17 kept its white text. The range test is written so a missing or non-numeric type never matches, and it follows your `new_types` list (continuous stretches become one range each).

4. Limits, gotchas and lessons

Limits

Limit	Why
<code>rareFlag</code> 14 to 49 for new types	Small texture ids $N+100$ / $N+300$ must stay below TOTW's 151 / 351.
16 new types	Space in the smallest background-switch region (fake big card). Going further needs a range-based rewrite.
4 colored types	Space in the footer-color region.

Gotchas

- **Restart after adding art.** During testing, a texture that was missing wasn't looked for again in the same session, so new files show up reliably only after a restart.
- **Missing art shows as a white rectangle.** Usually a file-name typo or a wrong folder.
- **Types not in `new_types`** (for example 30) still get EA's fallback: odd means rare look plus glare, even means common.
- **Server side:** the local server's own logic treats any rareFlag above 1 as "special" for things like pricing, independent of these UI edits.
- **Game updates or reinstalls:** if EA's UI movies ever change, the generator's SHA-256 check stops it (with an error at launch) rather than patching blind.
- **Installing needs the game closed and admin rights.** The launcher does both; running the installer by hand while FIFA runs is refused.
- **Don't hand-edit `fut-ui-byte-edits.txt`**. It's regenerated from the JSON on every launch.

Lessons from development (useful for future edits)

- **The same UI movie exists in three places** (patch.big, cards0.big, loose). Loading through the archives, patch.big wins. Keeping all three identical removes a whole class of "why didn't it change?" questions.
- **Most card logic exists twice:** a real class and a "fake" preview class, and `Card` (big) is separate from `CardSmall` (pitch). The pitch stayed unchanged for a while because the first patched small-card function turned out to be the fake one.
- **Native code is off-limits** (anti-tamper). Inline hooks crashed the game; hardware breakpoints were safe for short probes but shouldn't be left armed.
- **File-open tracing misleads after the first screen**, because the bulk pre-cache loads everything at once.
- **Retail debug strings are gold for orientation.** The movies still contain names like `Card::setBackgroundImage()` in their `trace()` calls, which is how functions were identified.

Appendix A: rareFlag to texture ids

"Your name" is how the types are used in this save. EA's internal enum names are shuffled against the designs, for example rareFlag 11 is TOTY_CARD in EA's code but carries the TOTS design. Tier = bronze / silver / gold.

rareFlag	Your name	EA enum	Large	Collapsed	Small (base / layer 2)
0 (even)	Common	(rarity COMMON)	1-3 by tier	1001-1003	101-103 / 301-303
1 (odd)	Rare	(rarity RARE)	11-13	1011-1013	111-113 / 311-313
3	TOTW	TOTW_CARD	51-53	1051-1053	151-153 / 351-353
4	Purple	PURPLE_CARD	60	1060	160 / 360
5	TOTY (blue)	BLUE_CARD	61	1061	161 / 361
6	Record Breaker (blue/red)	BLUERED_CARD	62	1062	162 / 362
7	Green	GREEN_CARD	63	1063	163 / 363
8	MOTM (orange)	ORANGE_CARD	64	1064	164 / 364
9	FUTTIES (pink)	PINK_CARD	65	1065	165 / 365
10	Teal	TEAL_CARD	66	1066	166 / 366
11	TOTS (tiered)	TOTY_CARD	67-69	1067-1069	167-169 / 367-369
12	Legend	LEGEND_CARD	70 (not shipped by EA)	1070	170 / 370
13	World Cup	WC_CARD	500	1500	600 / 800
14-28	Your new types	(none, raw value)	N	N+1000	N+100 / N+300
other 14-49	unassigned		EA fallback by parity (common or rare look). Free to add to new_types .		

Other ids in the large folder (8, 71-73, 81-83, 99 empty slot, 201-203, 211-213, 570, 700, 711-713, 800) belong to boosters, modifiers and special markers, not to player rareFlags.

Appendix B: all rewritten regions

Offsets are into the decompressed movie. Each region keeps its exact original size.

Movie	Function	Region	Purpose
card.big	Card::setBackgroundImage	0xF301-0xF4E4	Background switch (large, collapsed)
card.big	CardFake::_SetFakeBackground	0x90B1-0x924C	Background switch (fake big card)
cardsmall.big	CardSmall::setBackgroundImage	0x7BED-0x7DBC	Background switch (pitch)
cardsmall.big	CardSmall::_SetFakeBackground	0x5695-0x5830	Background switch (fake small card)
futmain.big	FutHelpers::_CacheItemBGImages	0x2FAC0-0x2FDA0	Texture pre-cache lists
card.big	Card::SetMainFontColors	0xD4E9-0xD785	Header and text colors (large, collapsed)
card.big	Card::setFooterFontColor	0xD855-0xDA2D	Stats row color (large)
cardsmall.big	CardSmall color function	0x75FE-0x7948	Rating and text colors (pitch)
card.big	Card::setBackgroundImage (start)	0xF169-0xF2CB	Shine off for new types
card.big	CardFake::_SetFakeBackground (end)	0x926B-0x92CE	Shine off (fake big card)
cardsmall.big	CardSmall::setBackgroundImage (start)	0x7AC1-0x7BDE	Shine off (pitch)
cardsmall.big	CardSmall::_SetFakeBackground (end)	0x585B-0x58BE	Shine off (fake small card)

Appendix C: opcode cheat sheet

Worked out from the files and confirmed against known behavior. "const8" means a 1-byte index into the code block's local name table.

Byte	Operand	Meaning	Byte	Meaning
A3	u16	push name (constant index)	59 / 5A	push 0 / push 1
A2	const8	push name	73 / 74	push true / false
AE	const8	get variable by name	75 / 76	push null / undefined
AF	const8	get member by name	1C / 1D	get / set variable
B0 / B2	const8	call named function / method, drop result	4E / 4F	get / set member
B5	i8	push byte (signed: use B6 for 128+)	3D	call function (name on stack, keeps result)
B6	u16	push 16-bit number	5B / 5C	EA call variants: drop / keep result
B7	u32	push 32-bit number (colors)	66	strict equals
B9	u8	push register	49	equals
87	u32, aligned	store register (value stays on stack)	48	less than (first pushed < second)
9D	s32, aligned	branch if true	67	greater than
99	s32, aligned	jump	12	not
8E	header, aligned	define function (registers). Code size at +16, body after an 8-byte marker	47	add
9B	header, aligned	define function (simple). Code size at +12	4C / 17	duplicate / pop
			26	trace (debug print)
			3E	return

Aligned operands: the 4-byte value starts at the next multiple of 4 after the opcode byte (0 to 3 zero padding bytes in between). Branch target = address of the operand + 4 + value.

Appendix D: commands and glossary

From a terminal in `C:\Games\FIFA-14-Local-FUT-main`. The launcher runs the first and third commands for you; run them by hand only to check something.

```
# Rebuild the edits from the config and show any error (safe, writes only the edits file)
python tools\generate_fifa14_card_slots.py

# Dry run: show what the installer would change (safe, writes nothing)
python tools\patch_fifa14_fut_ui_bytes.py --edits-file artifacts\fut-ui-byte-edits.txt

# Install now without launching (admin terminal, FIFA closed)
python tools\patch_fifa14_fut_ui_bytes.py --edits-file artifacts\fut-ui-byte-edits.txt --apply
```

Term	Meaning
rareFlag	Number on each item that tells the game its card type.
Apt / movie	EA's Flash-like UI format. A movie (<code>.big</code>) holds graphics and a script.
Bytecode / opcode	The script in compact binary form / one instruction's code byte.
Register	Numbered scratch variable inside a function (<code>r0</code> , <code>r1</code> ...).
Region	A stretch of a function that is rewritten at exactly its original size.
Archive / index / record	<code>patch.big</code> , <code>cards0.big</code> / their <code>.bh</code> tables of contents / one packed inner file.
Chunkzip	EA's compression of records in independent chunks.
DDS, DXT5	Texture file format and its compression mode (also called BC3).
Retail / original	The unmodified files EA shipped.
Fake card	EA's lighter card variant built straight from item data, used for previews.
Spinner patcher	An earlier project patch that rebuilds <code>patch.bh</code> on every launch.